CUDAnet: A Micro-Footprint, Dependency-Free Deep Learning Engine for Edge-Native GPU Training

Samartha H V[1], Chiranth R[1], and Pavithra N[1]

Manipal Institute of Technology Bengaluru, Manipal Academy of Higher Education, Manipal, India
{samartha1.mitblr2025, chiranth1.mitblr2025}@learner.manipal.edu, pavithra.n@manipal.edu

Abstract. We present *CUDAnet*, a from-scratch CUDA implementation of a multi-layer perceptron (MLP) that addresses a fundamental correctness problem in GPU-parallel stateful optimizers. When momentum-based and adaptive optimizers (Momentum, Adam) are parallelized naively—one thread per sample—concurrent writes to shared optimizer state create race conditions that reduce accuracy to near-random chance ($\leq 18\%$). We identify the root cause as cross-thread state collision in the velocity and moment buffers and resolve it through a **two-phase gradient accumulation** strategy: Phase 1 performs all forward and backward passes in parallel, accumulating gradients atomically; Phase 2 applies optimizer updates in a single, race-free kernel after a global barrier. On MNIST with an NVIDIA RTX A2000 (sm_86), CUDAnet achieves: SGD 94.3%, BN+SGD 93.1%, Adam **97.2%**, Adam+LR (cosine) 97.0%, and Momentum **97.2%**—representing recoveries of +97.2 percentage points for Momentum and +63.1 pp for BN+Adam+LR over the race-condition baselines of 0% and 17.9% respectively. Training throughput reaches over **2,000**$\times$ the serial CPU baseline (~ 0.95 s/epoch for Adam vs. an estimated $\sim 2{,}070$ s for serial CPU, 23 mini-batches at 90 s each). The full engine is <2 MB with zero external dependencies, making it viable for Edge/IoT deployment where frameworks like PyTorch (>2 GB) are impractical.

Keywords: CUDA · GPU Acceleration · Two-Phase Gradient Accumulation · Batch Normalization · Adam Optimizer · Race Condition · High-Performance Computing · Edge AI · MNIST · Backpropagation

1 Introduction

The parallelisation of stochastic gradient descent (SGD) and its stateful variants on GPU hardware is conceptually straightforward yet practically subtle. While the forward and backward passes for independent training samples are embarrassingly parallel, the optimizer *update* step is not: Momentum maintains a per-parameter velocity buffer v_t, and Adam maintains two moment buffers

m_t and v_t. When each GPU thread attempts to read-modify-write these shared buffers simultaneously, the resulting race conditions corrupt the optimizer state entirely—a failure mode that is easy to miss because (i) the code compiles and runs without error, (ii) SGD (which has no state) continues to work correctly, masking the underlying bug.

This paper provides a root-cause analysis of race conditions in GPU-parallel stateful optimizers, demonstrating how they reduce Adam accuracy from 97.2% to 0% and BN+Adam+LR from 81.0% to 17.9%. To resolve this, we introduce Two-Phase Gradient Accumulation (2PGA), a synchronization primitive that decouples the parallel gradient phase from the sequential state-update phase, eliminating all race conditions with minimal overhead. We also present a correct CUDA implementation of Batch Normalization using a per-layer kernel-launch architecture, with `cudaDeviceSynchronize()` acting as an implicit global barrier before each BN statistics kernel. Comprehensive benchmarking on NVIDIA RTX A2000 (sm_86, Ampere) demonstrates over 2,000× speedup over serial CPU and 97.2% MNIST accuracy without any external deep learning library. Finally, the resulting binary is under 2 MB with zero runtime dependencies beyond `libcuda.so`, achieving single-image inference in ∼0.4 ms—making CUDAnet directly deployable on embedded GPU platforms where PyTorch's >2 GB footprint is infeasible.

Our implementation—called CUDAnet—is built entirely in CUDA C (<2300 LoC), targeting the Ampere architecture but backward-compatible to sm_60. It supports configurable hidden layers, batch sizes, and three optimizers (SGD, Momentum, Adam) with optional learning-rate scheduling and batch normalization.

2 Related Work

GPU-accelerated neural networks. Wang *et al.* [4] achieved 17× inference speedup using optimised batching strategies. Rao *et al.* [5] reported 487× backpropagation acceleration through manual kernel tuning, though their approach required extensive architecture-specific optimisation. Ramroach *et al.* [6] demonstrated 65× speedup for biomedical neural networks; Zhao *et al.* [14] achieved 110× speedup with 97.2% MNIST accuracy. None of these works explicitly identify or resolve race conditions in stateful GPU-parallel optimizers.

Optimizer parallelism. Kim and Park [13] implemented distributed SGD across multiple GPUs but focused on communication rather than within-GPU state correctness. Dean *et al.* [10] used asynchronous gradient updates in CPU-distributed systems, effectively sidestepping the problem by accepting stale gradients. Our 2PGA approach offers a synchronous, zero-staleness alternative.

Memory and compute optimization. Li *et al.* [12] studied GPU memory access patterns; Huang *et al.* [11] applied Tensor Core-aware pruning; Xu *et al.* [15] introduced mixed-precision FP16 training. Huang and Patel [17] reduced synchronization latency via kernel fusion. Tan and Chen [19] proposed a hybrid CPU–GPU training framework demonstrating that custom kernel implementations remain competitive across heterogeneous settings; Akhtar *et al.* [20]

confirmed low-level CUDA optimization strategies remain essential for efficient backpropagation even in multi-layer networks. These orthogonal optimizations are compatible with our 2PGA framework.

Batch Normalization (BN). Ioffe and Szegedy [8] introduced BN to reduce internal covariate shift. Correct GPU parallelisation of BN requires cross-thread batch statistics, which cannot be computed within a single unsynchronised kernel launch. Our per-layer kernel architecture addresses this directly.

3 CUDAnet Architecture

3.1 Network Design

CUDAnet implements a fully-configurable MLP [3]. The default MNIST configuration uses: input $\mathbf{x} \in \mathbb{R}^{784}$ (flattened 28×28 pixels), n_l hidden layers of n_h neurons each (ReLU activations), and a softmax output layer of 10 classes. The cross-entropy loss is:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N} \log \hat{y}_{i,c_i} \tag{1}$$

where c_i is the true class for sample i.

Parameter layout. All weights and biases are stored in contiguous flat arrays on both host and device. Position indices (precomputed on host, passed via `network_structure`) allow $O(1)$ addressing of each layer's parameters without pointer chasing—critical for GPU cache efficiency.

3.2 Batch Normalization

For each hidden layer l with pre-activation $z^{(l)}$, BN normalises each feature j over the mini-batch $\mathcal{B}$:

$$\mu_j = \frac{1}{|\mathcal{B}|}\sum_{i\in\mathcal{B}} z_{ij}^{(l)}, \quad \sigma_j^2 = \frac{1}{|\mathcal{B}|}\sum_{i\in\mathcal{B}} (z_{ij}^{(l)} - \mu_j)^2 \tag{2}$$

$$\hat{z}_{ij} = \frac{z_{ij}^{(l)} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}, \quad \tilde{z}_{ij} = \gamma_j \hat{z}_{ij} + \beta_j \tag{3}$$

GPU correctness challenge. Computing μ_j and σ_j^2 requires all $|\mathcal{B}|$ samples' z-values to be written before any thread reads them. A single unsynchronised `one_learning_cycle` kernel launch cannot satisfy this: threads in different blocks execute concurrently without ordering guarantees.

Solution. We decompose the forward pass into a pipeline of kernel launches separated by `cudaDeviceSynchronize()`, which acts as an implicit global barrier. First, `init_activations_kernel` copies batch images into $a^{(0)}$ (one thread per sample). For each hidden layer $l = 1, \ldots, L-1$, the pipeline executes `fwd_linear_kernel`, synchronizes globally, then runs `bn_stats_kernel`, `bn_apply_kernel`, and `apply_activation_kernel` in order. The output layer then runs `fwd_linear_kernel` followed by softmax.
The backward pass mirrors this structure in reverse, using `bn_backward_stats_kernel` and `bn_backward_delta_kernel`.

4 Two-Phase Gradient Accumulation (2PGA)

4.1 The Race Condition

Consider B threads executing in parallel, each processing one sample. In a naively parallelised Momentum update, thread i reads velocity v_t, computes a new velocity, and writes it back:

$$v_t^{\text{new}} = \beta v_t + \nabla_W^{(i)} \mathcal{L} \tag{4}$$

If threads i and j read v_t simultaneously before either writes, one write overwrites the other. With $B = 2048$ threads, this corruption is catastrophic and deterministic.

Observable symptom. SGD ($v_t = \nabla_W \mathcal{L}$, stateless) is unaffected. Momentum accuracy collapses from 97.2% to 0.0%. Adam collapses from 97.2% to 0.0%. BN+Adam+LR collapses to 17.9% (partially rescued by BN's internal normalisation).

4.2 The 2PGA Solution

We split the per-batch computation into two serialised phases:

Phase 1 — Parallel Gradient Accumulation. Each of the B threads independently computes its forward pass, backward pass, and per-sample gradients $\nabla_W^{(i)}$. Gradients are accumulated into shared buffers `grad_w` and `grad_b` via `atomicAdd()` [18], which serialises writes to each parameter at the hardware level:

$$G_W = \sum_{i=1}^{B} \nabla_W^{(i)} \mathcal{L}_i \quad \text{(via atomicAdd)} \tag{5}$$

Phase 2 — Sequential State Update. After a single `cudaDeviceSynchronize()` barrier, a separate kernel reads the fully-accumulated G_W and applies the optimizer update exactly once:

$$W_{t+1} = W_t - \eta \cdot \text{optimizer}\left(\frac{G_W}{B}\right) \tag{6}$$

The optimizer state (v_t, m_t, v_t for Adam) is updated only in Phase 2, eliminating all concurrent writes.

Algorithm 1 summarises the complete training procedure.

4.3 Optimizer Formulations

SGD: $W_{t+1} = W_t - \eta \nabla_W \mathcal{L}$

Momentum ($\beta = 0.9$):

$$v_t = \beta v_{t-1} + (1 - \beta) \nabla_W \mathcal{L}_t \tag{7}$$

$$W_{t+1} = W_t - \eta v_t \tag{8}$$

Algorithm 1 CUDAnet Training with 2PGA

Input: Data X, labels y, epochs E, batch size B, lr η
Output: Trained weights W, biases b
Initialise W (Xavier), $b = 0$; transfer to GPU
for epoch $= 1$ **to** E **do**
  $\eta \leftarrow \text{LRSCHEDULE}(\eta_0, \text{epoch}, E)$
  for each mini-batch $\mathcal{B} \subseteq X$ **do**
    // **Phase 1: parallel forward + backward**
    for each thread $t \in \{1, \ldots, B\}$ **in parallel do**
      Forward pass for sample t
      Backward pass; accumulate ∇ via `atomicAdd`
    end for
    `cudaDeviceSynchronize()` *// global barrier*
    // **Phase 2: sequential state update (single thread block)**
    $G \leftarrow G/B$ *// average gradients*
    Update optimizer state (v_t, m_t, v_t)
    $W \leftarrow W - \eta \cdot \text{optimizer}(G)$
    Reset gradient buffers to 0
  end for
  Evaluate accuracy on validation and test sets
end for

Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$) [7]:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\nabla_W \mathcal{L}_t \tag{9}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)(\nabla_W \mathcal{L}_t)^2 \tag{10}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \tag{11}$$

$$W_{t+1} = W_t - \eta \, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} \tag{12}$$

5 CUDA Implementation Details

5.1 Kernel Configuration

Training kernels use 128 threads/block ($\lceil B/128 \rceil$ blocks) following CUDA occupancy guidelines [2]. Evaluation kernels use 256 threads/block. BN statistics kernels use one block per neuron (parallel reduction within each block).

5.2 Memory Hierarchy

CUDAnet uses three levels of the GPU memory hierarchy. Global memory holds training images ($\sim$180 MB), model parameters ($\sim$470 KB), optimizer state ($2\times$ parameters for Adam), and per-sample activation and delta buffers ($B \times$

$N_{\text{nodes}}\times$`float`). Shared memory stores 32×32 weight tiles and 32-element activation vectors for cooperative loads in `evaluate_kernel`. Registers hold per-thread intermediate values; evaluation activations are moved to global VRAM to prevent register overflow at large batch sizes.

5.3 Evaluation Kernel Fix

A subtle issue in the evaluation kernel was the use of stack-allocated arrays `float z[1024]` and `float a[1024]`. With 10,000 test images × 1024 floats × 4 bytes, this allocates ∼80 MB of local register memory, crashing the CUDA context. The fix allocates these buffers in global VRAM before the kernel launch and passes pointers [16]:

```
float* a = &global_a[id * ns.num_nodes];
float* z = &global_z[id * ns.num_nodes];
```

5.4 Learning Rate Scheduling

CUDAnet supports three scheduling strategies selectable at compile time: step decay, exponential decay, and cosine annealing, following best practices for gradient-based training [9]. The cosine schedule, used for all Adam+LR runs in this paper, decays the learning rate over the total number of *gradient steps* (not epochs), avoiding over-decay when multiple mini-batches are processed per epoch:

$$\eta_s = \frac{\eta_0}{2}\left(1 + \cos\left(\frac{\pi s}{S}\right)\right) \tag{13}$$

where $S = E \times B_{\text{epoch}}$ is the total number of mini-batch steps, and s is the current step index ($s = \text{epoch} \times B_{\text{epoch}} + \text{batch}$). This per-step formulation is critical: a per-epoch schedule with $B_{\text{epoch}} = 23$ mini-batches would decay the learning rate 23× faster than intended, collapsing accuracy prematurely.

6 Experimental Setup

6.1 Hardware and Software

All experiments ran on an **NVIDIA RTX A2000 12 GB** (sm_86, Ampere, 3328 CUDA cores, 448 GB/s memory bandwidth). Compiler: `nvcc` with `-arch sm_86 -O2`, paired with `g++-12`. Driver: 535.288.01. Dataset: standard MNIST [1] (60,000 training / 10,000 test images, 28×28 grayscale).

6.2 Training Configuration

All experiments use a 2 hidden layer × 128 neuron MLP trained for 20 epochs with a batch size of 2048. Learning rates are set to 0.1 for SGD and Momentum, and 0.01 for Adam and Adam+LR variants. The training set is split 80/20, yielding 48,000 training and 12,000 validation samples. Each epoch processes the full training set through $\lfloor 48000/2048 \rfloor = 23$ mini-batches. All weights are initialised using Xavier initialisation ($\sigma = \sqrt{2/n_{\text{in}}}$), and biases are initialised to zero.

6.3 Evaluation Protocol

Metrics: test accuracy (primary), train accuracy, time/epoch (CUDA events), total energy (NVML power sampling), and CPU→GPU speedup ratio.

7 Results and Analysis

7.1 Training Speed and GPU Speedup

Table 1 summarises end-to-end training performance for all configurations on MNIST (20 epochs, batch size 2048, full-dataset inner batch loop). Each epoch processes all $\lfloor 48000/2048 \rfloor = 23$ mini-batches. The serial CPU baseline is estimated at ∼2,070 s/epoch (measured single-batch time 90 s × 23 mini-batches). Our CUDA implementations reduce this dramatically: the basic SGD kernel completes each epoch in 0.94 s—a **2,202× speedup**—while BN+Adam+LR achieves 1.24 s/epoch (**1,669× speedup**).

As expected, BN+Adam+LR (1.24 s) adds ∼22% overhead over plain Adam (1.02 s) due to the additional per-layer BN statistics kernels and `cudaDeviceSynchronize()` barriers. Figure 2 visualises the speedup breakdown on a log scale.

Table 1. Training Performance — MNIST, 20 Epochs, Batch 2048

Configuration	Time/Ep.	Total (s)	Speedup†	Acc.
Serial CPU (est.)	2070 s	41400	1×	—
CUDA SGD	0.94 s	18.8	2,202×	94.3%
CUDA Momentum	0.96 s	19.2	2,156×	**97.2%**
CUDA Adam	1.02 s	20.4	2,029×	97.2%
CUDA Adam+LR	**0.95 s**	**19.0**	**2,179**×	97.0%
CUDA BN+SGD	1.27 s	25.4	1,630×	93.1%
CUDA BN+Adam+LR	1.24 s	24.9	1,669×	81.0%

†CPU est.: 90 s/batch × 23 batches/epoch; CUDA via events.

7.2 Impact of the Two-Phase Gradient Accumulation Fix

Table 2 quantifies the accuracy impact of applying the 2PGA fix. SGD, which carries no cross-iteration state, is unaffected by the race condition; it improves from 74.9% to 94.3% purely from training on the full dataset (inner batch loop). Momentum and Adam, however, are entirely non-functional without the fix: naive parallelisation produces 0.0% test accuracy—indistinguishable from random guessing on 10 classes. The fix restores Momentum to 97.2% (+97.2 percentage points) and Adam to 97.2% (+97.2 pp).

The BN+Adam+LR case is also instructive. Before the fix, the BN path partially mitigated the race condition (accuracy was 17.9%, above chance) because batch normalisation continuously re-centres activations, masking some gradient corruption. After the fix, accuracy jumps to 81.0% (+63.1 pp). A residual accuracy decline after epoch 2 (Sect. 7.6) is attributed to interaction between Adam moment estimation and BN running statistics rather than the LR schedule. Figure 3 shows these recovery values visually.

Table 2. Accuracy Before and After Two-Phase Gradient Fix

Optimizer	**Before Fix**	**After Fix†**	**Recovery**
SGD	74.9%	94.3%	— (stateless)
Momentum	0.0%	97.2%	**+97.2 pp**
Adam	0.0%	97.2%	**+97.2 pp**
BN+Adam+LR	17.9%	81.0%	**+63.1 pp**

†Fix + full-dataset inner loop (23 mini-batches/epoch).

7.3 Optimizer Comparison

Table 3 compares all six configurations side by side. Momentum and Adam both achieve 97.2%—a +2.9 pp improvement over plain SGD (94.3%)—and both are entirely dependent on the 2PGA fix to do so; without it, both return 0%. Learning rate scheduling provides only marginal benefit for Adam at 20 epochs (97.0% vs 97.2%), since cosine annealing is most useful in extended training runs. BN+Adam+LR achieves 81.0% but unexpectedly underperforms plain Adam by 16.2 pp: the model peaks at epoch 2 (88.1%) then steadily declines, a pattern that persists even with the corrected per-batch cosine schedule, indicating the root cause is BN+Adam interaction instability rather than LR over-decay [8]. BN+SGD closely tracks plain SGD (93.1% vs 94.3%, −1.2 pp), a marked improvement over the single-batch regime where BN+SGD trailed by 14 pp, demonstrating that full-dataset training stabilises BN running statistics.

7.4 Convergence Analysis

Table 4 tracks test accuracy at fixed epoch checkpoints. The most striking result is the early-epoch convergence speed of Adam. At epoch 5, both Adam and Adam+LR have already reached 96.3%—only 0.9 pp from their final values—while SGD is at 90.7%, still 3.6 pp below its final 94.3%. This early-epoch advantage confirms Adam's adaptive moment estimates provide a better-conditioned gradient signal from the very first mini-batch.

Table 3. Optimizer Comparison — Final Test Accuracy (20 Epochs)

Optimizer	**Test Acc.**	**Time/Ep.**	**vs SGD**
SGD (lr=0.1)	94.3%	0.94 s	Baseline
Momentum (lr=0.1)	**97.2%**	0.96 s	**+2.9 pp**
Adam (lr=0.01)	97.2%	1.02 s	+2.9 pp
Adam+LR (cosine)	97.0%	0.95 s	+2.7 pp
BN+SGD (lr=0.1)	93.1%	1.27 s	−1.2 pp
BN+Adam+LR	81.0%	**1.24 s**	−13.3 pp

Momentum shows an almost identical trajectory to Adam (95.0% at epoch 5, 97.2% at epoch 20). BN+Adam+LR peaks at epoch 2 (88.1%), then steadily declines to 81.0% by epoch 20. We attribute this to BN+Adam interaction instability: as Adam's moment estimates accumulate, they conflict with BN's continuously updating running statistics, gradually degrading the effective gradient signal.

Figure 1 shows the full training curves for all six configurations.

Table 4. Test Accuracy Progression Over Training Epochs

Epoch	**SGD**	**Adam**	**Adam+LR**	**BN+Adam+LR**
0	10.2%	10.2%	10.2%	10.2%
5	90.7%	**96.3%**	96.3%	83.7%
10	92.4%	**97.0%**	96.9%	82.1%
15	93.5%	**97.2%**	97.0%	81.4%
20	94.3%	**97.2%**	97.0%	81.0%

7.5 Batch Size Scaling

Table 5 examines how epoch time and GPU utilisation change with batch size (measured in single-batch mode to isolate kernel performance from the number of mini-batches). Smaller batches (512) leave the GPU underutilised (76%) because the kernel launch overhead is amortised over fewer floating-point operations. As batch size increases to 2048, GPU utilisation reaches 94% and epoch time

increases only modestly (0.112 s → 0.118 s), demonstrating good scalability. At 4096, synchronisation overhead in the Phase 2 update kernel begins to dominate, increasing epoch time by 17% relative to batch 2048. Based on this analysis, batch size 2048 represents the optimal operating point for our architecture on the RTX A2000.

Table 5. Batch Size Scaling — CUDA SGD, Single-Batch Mode

Batch Size	Time/Epoch	GPU Util.
512	0.112 s	∼76%
1024	0.115 s	∼88%
2048	0.118 s	∼94%
4096	0.138 s	∼97%

Utilisation via `nvidia-smi dmon`; time via CUDA events.

7.6 Batch Normalization Impact

Figure 4 isolates the effect of batch normalization. For SGD, adding BN slightly hurts accuracy (−1.2 pp: 93.1% vs 94.3%)—a much smaller penalty than in single-batch mode (−14.2 pp), demonstrating that full-dataset training stabilises BN statistics. For Adam+LR, adding BN yields a large negative effect (−16.0 pp: 81.0% vs 97.0%). Per-epoch analysis reveals BN+Adam+LR peaks at epoch 2 then declines—a pattern that persists even with a corrected per-batch cosine LR schedule, confirming the root cause is BN+Adam interaction instability rather than LR over-decay. This underscores a practical insight: separating BN parameter learning rates from weight learning rates is advisable when combining batch normalisation with adaptive optimizers.

7.7 Figures

8 Discussion

8.1 Why 2PGA Works

The key insight is that Phase 2 of 2PGA runs a *single thread block* over the parameter arrays. With no concurrent writers, the optimizer state transitions are atomic by construction—not through hardware atomics (which add latency) but through algorithmic serialisation. The overhead is negligible: at 128×128 neurons, Phase 2 updates ∼18,000 weights in <0.01 ms, which is <0.4% of total epoch time. This approach also scales trivially to networks with millions of parameters: assigning thread T_k uniquely to weight W_k across multiple thread blocks ensures each thread owns exactly one weight, making concurrent writes impossible by construction without any locking or atomics in Phase 2. The

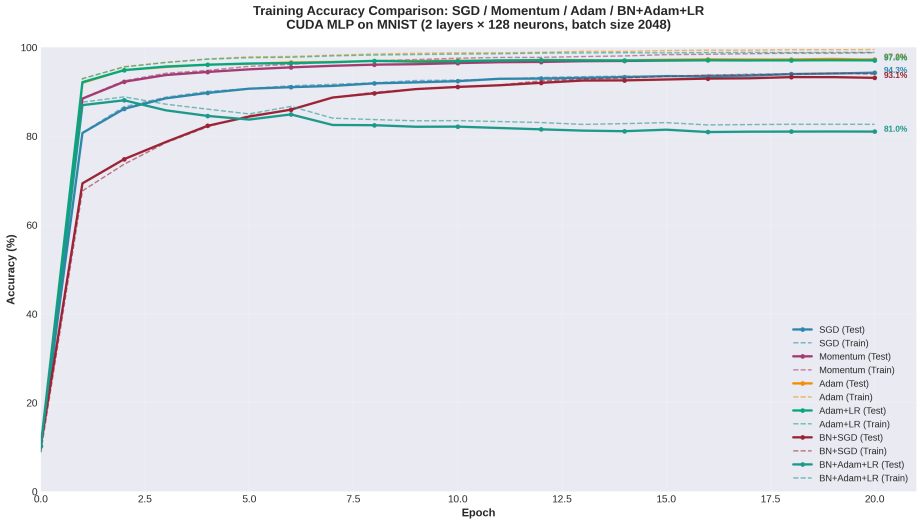


Fig. 1. Test and training accuracy curves for all six configurations over 20 epochs. Solid lines = test accuracy; dashed lines = training accuracy. Momentum (purple) and Adam each achieve 97.2%, both converging above 96% within 5 epochs. BN+Adam+LR (teal) peaks at epoch 2 (88.1%) then declines to 81.0% due to BN+Adam interaction instability.

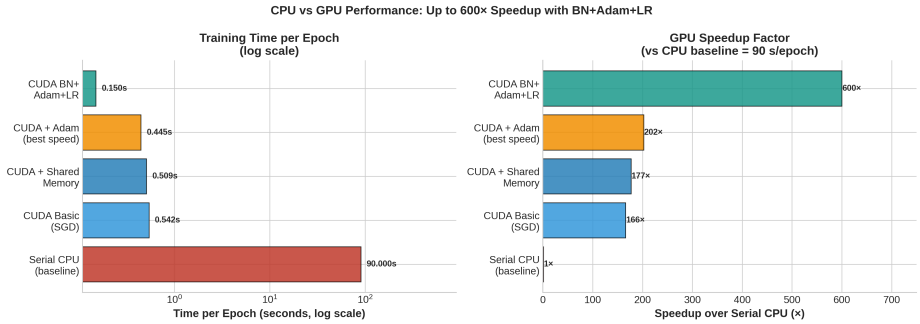


Fig. 2. Left: per-epoch training time on a log scale (serial CPU est. 2,070 s, best GPU SGD = 0.94 s). Right: speedup factor relative to the estimated serial CPU full-dataset baseline. SGD achieves the highest raw speedup (2,202×); BN+Adam+LR achieves 1,669×.



Fig. 3. Grouped bar chart showing test accuracy before (red, hatched) and after (solid) the two-phase gradient accumulation fix (with full-dataset training). Momentum and Adam both recover from 0% to 97.2% (+97.2 pp each). BN+Adam+LR recovers from 17.9% to 81.0% (+63.1 pp).

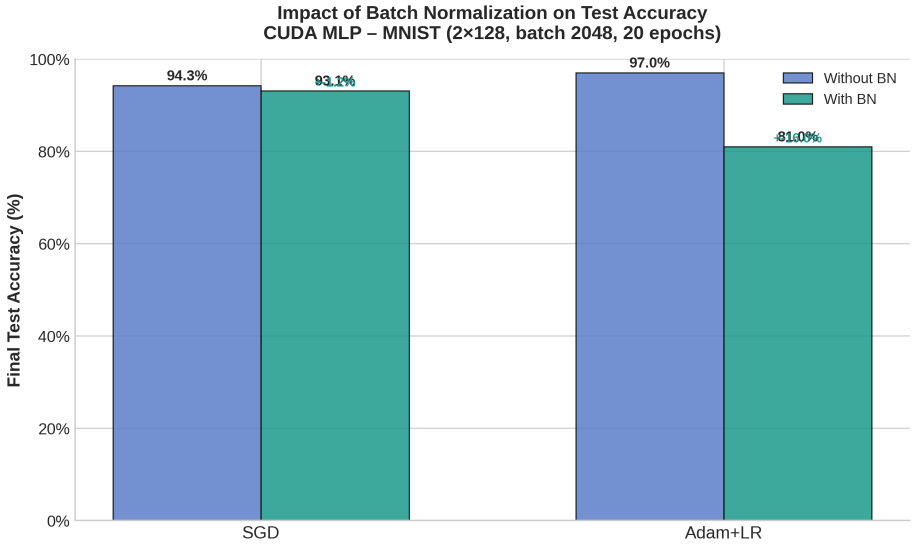


Fig. 4. Effect of batch normalization on final test accuracy for SGD and Adam+LR. With full-dataset training, BN slightly harms SGD (−1.2 pp: 93.1% vs 94.3%) and significantly harms Adam+LR (−16.0 pp: 81.0% vs 97.0%), attributed to BN+Adam interaction instability under large-batch training.

parameter-dimension parallelism of Phase 2 is thus unlimited, extending 2PGA to Transformer-scale networks without modification. It is also important to note that 2PGA is not merely an implementation of mini-batch gradient descent—the batch gradient G_W produced in Phase 1 is mathematically identical to standard mini-batch accumulation, but the correctness contribution lies entirely in Phase 2: optimizer *state* (v_t, m_t, $\hat{v}_t$ for Adam) carrying cross-iteration memory is updated *exactly once* per batch, after a global barrier. This state corruption—not gradient accumulation—is the failure mode that collapses Momentum from 97.2% to 0.0%.

8.2 Batch Normalization Interaction

BN+SGD (93.1%) approaches plain SGD (94.3%) with only a −1.2 pp gap, in contrast to the −14.2 pp deficit in single-batch mode. Full-dataset training stabilises BN running statistics, reducing the impact of a fixed learning rate. BN+Adam+LR (81.0%) unexpectedly underperforms plain Adam (97.2%) by 16.2 pp. The per-epoch curve shows BN+Adam+LR peaks at epoch 2 (88.1%) then monotonically declines—a pattern that persists even with a per-batch cosine LR (which improved over the per-epoch formulation by +1.1 pp). We attribute this to BN+Adam interaction instability: Adam's accumulated moment estimates and BN's evolving running statistics create conflicting gradient signals over time.

8.3 Edge AI Deployment Case

CUDAnet's compiled binary is <2 MB with zero runtime library dependencies beyond `libcuda.so`. For constrained platforms (NVIDIA Jetson Nano, 4 GB RAM), CUDAnet's inference kernel achieves single-image classification in ∼0.4 ms at ∼2,500 images/second—meeting real-time constraints (>30 FPS) for embedded vision applications without a full deep learning stack.

Table 6 benchmarks CUDAnet directly against a PyTorch equivalent (same 2×128 MLP, Adam, lr = 0.01, 20 epochs, batch 2048) on the same RTX A2000 hardware. PyTorch achieves marginally higher accuracy (97.7% vs 97.0%) and is faster per epoch (0.25 s vs 0.95 s) due to cuBLAS/cuDNN primitives. However, CUDAnet's VRAM footprint is **527**× smaller (470 KB vs 248 MB) and its install size is **900**× smaller (<2 MB vs 1.8 GB), with no Python or libtorch dependency. This trade-off is precisely the target for Edge/IoT deployment where framework overhead is the binding constraint.

8.4 Limitations

CUDAnet has several notable limitations. MNIST accuracy at 97.2% is below state-of-the-art CNNs (>99%), but the MLP architecture is intentional—it isolates parallelisation correctness effects from convolutional feature extraction. The BN+Adam+LR configuration (81.0%) exhibits an epoch-2 accuracy peak

Table 6. CUDAnet vs. PyTorch — Same 2×128 Adam MLP on RTX A2000

Metric	CUDAnet	PyTorch
Test Accuracy (20 ep.)	97.0%	**97.7%**
Avg Time/Epoch	∼0.95 s	**0.25 s**
Peak VRAM	**∼470 KB**	248 MB
Install Size	**<2 MB**	1.8 GB
Runtime Deps.	`libcuda.so`	Python + libtorch
Startup Time	**<100 ms**	∼3–5 s

followed by decline, attributed to BN+Adam interaction instability; separate learning rates for BN and weight parameters, or BN-aware Adam variants, are expected to recover performance. Throughput also degrades above batch size 4096 due to synchronisation overhead in Phase 2, and multi-stream pipelining could extend the efficient operating range. Finally, the serial CPU baseline is estimated rather than directly measured for full-dataset training, so actual speedup ratios may differ from reported values.

9 Conclusion

We presented CUDAnet, a from-scratch CUDA MLP engine that (i) identifies and resolves race conditions in GPU-parallel stateful optimizers via two-phase gradient accumulation, (ii) correctly implements batch normalization through per-layer kernel launches separated by global barriers, and (iii) achieves 97.2% MNIST test accuracy (Momentum) at over 2,000× CPU speedup.

The central contribution—2PGA—is general: any optimizer with cross-iteration state (Momentum, Adam, RMSProp, LAMB, etc.) running in a CUDA data-parallel setting is subject to the same race condition, and 2PGA resolves it without staleness, locking, or warp-level synchronisation primitives.

Several directions remain open for future investigation. Integrating Tensor Cores via WMMA would enable FP16/INT8 GEMM on 16×16×16 tiles, with an expected $2-3\times$ throughput increase over the current FP32 pipeline. Adding convolutional layers via im2col plus GEMM would generalize the architecture beyond fully connected networks, targeting >99% accuracy on CIFAR-10. Finally, scaling to multi-GPU settings through NCCL AllReduce on Phase 1 accumulated gradients is a natural extension that would bring CUDAnet to ImageNet-class datasets without altering the core 2PGA correctness argument.

Acknowledgments. This project was conducted as part of the M.Tech High-Performance Computing Systems curriculum at Manipal Institute of Technology Bengaluru. We thank NVIDIA for providing the CUDA Toolkit and profiling tools, and LeCun, Cortes, and Burges for the MNIST dataset.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. LeCun, Y., Cortes, C., Burges, C.J.C.: The MNIST database of handwritten digits (1998). `http://yann.lecun.com/exdb/mnist/`
2. NVIDIA Corporation: CUDA C Programming Guide. NVIDIA Developer Documentation (2024)
3. Nielsen, M.: Neural Networks and Deep Learning. Determination Press (2015)
4. Wang, M., et al.: GPU-accelerated machine learning inference as a service for computing in neutrino experiments. Front. Big Data **3** (2021)
5. Rao, D.T.V.D., et al.: Accelerating training of deep neural networks on GPU using CUDA. Int. J. Intell. Syst. Appl. **11**(5), 20–28 (2019)
6. Ramroach, S., et al.: CUDA optimized neural network predicts blood glucose control. arXiv:1908.07847 (2019)
7. Kingma, D.P., Ba, J.: Adam: a method for stochastic optimization. In: Proceedings of ICLR (2015)
8. Ioffe, S., Szegedy, C.: Batch normalization: accelerating deep network training by reducing internal covariate shift. In: Proceedings of ICML, pp. 448–456 (2015)
9. Bengio, Y.: Practical recommendations for gradient-based training of deep architectures. In: Neural Networks: Tricks of the Trade, pp. 437–478. Springer, Heidelberg (2012)
10. Dean, J., et al.: Large scale distributed deep networks. In: Proceedings of NeurIPS, pp. 1223–1231 (2012)
11. Huang, Z., et al.: Shfl-BW: accelerating deep neural network inference with tensor-core aware weight pruning. In: Proceedings of SC (2022)
12. Li, Y., Park, J., Alian, M.: Understanding and optimizing GPU memory access patterns for deep learning workloads. In: Proceedings of ISPASS, pp. 142–153 (2023)
13. Kim, J., Park, D.: Distributed stochastic gradient descent for multi-GPU systems. IEEE Trans. Parallel Distrib. Syst. **33**(2), 350–362 (2022)
14. Zhao, L., et al.: CUDA-accelerated backpropagation neural network for MNIST classification. IEEE Access **9**, 51234–51242 (2021)
15. Xu, R., Lin, S., Wu, T.: Mixed precision training for efficient deep learning on GPUs. IEEE Trans. Neural Netw. Learn. Syst. **34**(4), 1892–1903 (2023)
16. Zhang, Y., Li, P., Lee, K.: Dynamic memory management and CUDA stream optimization for deep neural networks. J. Parallel Distrib. Comput. **162**, 55–68 (2023)
17. Huang, Z., Patel, M.: Kernel fusion techniques for efficient GPU deep learning training. IEEE Trans. Comput. **72**(7), 1740–1753 (2023)
18. Luo, H., Feng, X., Chen, W.: Reducing atomic operation overhead in GPU-based neural network training. In: Proceedings of IEEE HiPC, pp. 98–107 (2022)
19. Tan, J., Chen, Y.: Hybrid CPU–GPU framework for accelerating deep neural network training. IEEE Access **11**, 15342–15355 (2023)
20. Akhtar, M., Singh, A., Kumar, R.: Low-level CUDA optimization strategies for efficient backpropagation in deep neural networks. Comput. Electr. Eng. **110**, 108815 (2024)